

Laporan Praktikum Mandiri K-Nearest Neighbor (KNN)”

Nama Mahasiswa : Muhamad Faqih Rayya

Program studi : Teknik Informatika STT Terpadu Nurul Fikri, Depok

E-mail : 0110224212@student.nurulfikri.ac.id

Link Github : <https://github.com/faqihrayya0403/Machine-Learning>

Abstrak

Laporan ini membahas implementasi algoritma K-Nearest Neighbor (KNN) dalam melakukan klasifikasi pada data kelulusan mahasiswa. Tahapan yang dilakukan meliputi preprocessing data, encoding variabel kategorikal, normalisasi, penyeimbangan data menggunakan SMOTE, pemilihan hyperparameter dengan GridSearchCV, hingga evaluasi performa model menggunakan accuracy, precision, recall, dan confusion matrix.

1. Import Library

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GridSearchCV
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score, precision_score, recall_score
from imblearn.over_sampling import SMOTE
```

Penjelasan:

Bagian ini mengimpor seluruh library yang digunakan: Pada bagian ini, kamu mengimpor semua library yang dibutuhkan untuk proses *machine learning* menggunakan algoritma **K-Nearest Neighbors (KNN)**, termasuk *preprocessing*, *oversampling*, *hyperparameter tuning*, dan evaluasi model.

2. Mengakses File di Google Drive

```
from google.colab import drive
drive.mount('/content/gdrive')
```

... Drive already mounted at /content/gdrive; to attempt to forcibly remount, call drive.mount("/content/gdrive", force_remount=True).

Penjelasan

Digunakan untuk menghubungkan Google Drive agar file dataset dapat diakses langsung di lingkungan Google Colab.

3. Mengunggah File ke Google Colab

```
from google.colab import files
files.upload()
```

Penjelasan: Perintah ini digunakan untuk mengimpor modul files dari Google Colab.

Modul ini menyediakan fungsi untuk mengupload dan mendownload file antara komputer lokal kamu dan lingkungan Google Colab.

4. Membaca dan Membersihkan Dataset

```
train = pd.read_excel("kelulusan_test.xls")
test = pd.read_excel("kelulusan_train.xls")

# Hapus spasi pada nama kolom
train.columns = [c.strip() for c in train.columns]
test.columns = [c.strip() for c in test.columns]
```

Penjelasan:

- Perintah ini digunakan untuk **membaca file Excel** bernama *kelulusan_test.xls* lalu menyimpannya sebagai DataFrame bernama **train**.

Dataset *train* biasanya berisi data yang digunakan untuk melatih (training) model.

- - Mengambil semua nama kolom dalam dataset **train**
- Menghapus spasi di **awal** dan **akhir** nama kolom menggunakan `.strip()`

Ini penting agar:

- Nama kolom tidak menyebabkan error

- Kolom bisa dipanggil tanpa masalah

5. Menghapus Kolom yang Tidak Dibutuhkan

```
for col in ["NAMA", "STATUS NIKAH"]:  
    if col in train.columns:  
        train = train.drop(columns=[col])  
    if col in test.columns:  
        test = test.drop(columns=[col])
```

Penjelasan: Kode ini digunakan untuk **menghapus kolom yang tidak diperlukan** dalam proses analisis atau pelatihan model machine learning.

6. . Mengubah Label STATUS KELULUSAN Menjadi Data Numerik

```
train["STATUS KELULUSAN"] = train["STATUS KELULUSAN"].map(  
    lambda x: 0 if str(x).strip().upper().startswith("TEPAT") else 1  
)  
  
test["STATUS KELULUSAN"] = test["STATUS KELULUSAN"].map(  
    lambda x: 0 if str(x).strip().upper().startswith("TEPAT") else 1  
)
```

Penjelasan: Bagian ini digunakan untuk **mengubah data kategori (teks)** pada kolom *STATUS KELULUSAN* menjadi **label numerik**, supaya bisa digunakan oleh model KNN. Model machine learning **tidak bisa memproses teks langsung**, sehingga perlu di-*encode*.

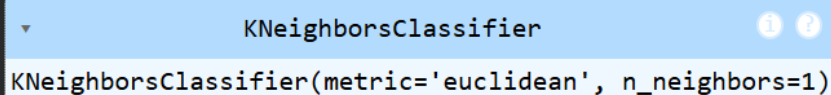
7. Mencari Hyperparameter Terbaik dengan GridSearchCV

```
param_grid = {  
    "n_neighbors": [1, 3],  
    "weights": ["uniform"],  
    "metric": ["euclidean"]  
}  
  
grid = GridSearchCV(KNeighborsClassifier(), param_grid, cv=3, scoring="accuracy")  
grid.fit(X_res, y_res)  
  
print("Best Params:", grid.best_params_)  
print("Best CV Score:", grid.best_score_)  
  
... Best Params: {'metric': 'euclidean', 'n_neighbors': 1, 'weights': 'uniform'}  
Best CV Score: 0.992831541218638
```

Penjelasan: Bagian ini digunakan untuk **mencari kombinasi parameter terbaik** (hyperparameter tuning) untuk algoritma **K-Nearest Neighbors (KNN)** menggunakan **GridSearchCV**.

8. Membangun Model KNN dengan Hyperparameter Terbaik

```
best_knn = KNeighborsClassifier(  
    n_neighbors=grid.best_params_["n_neighbors"],  
    metric=grid.best_params_["metric"],  
    weights=grid.best_params_["weights"]  
)  
  
best_knn.fit(X_res, y_res)
```



```
KNeighborsClassifier  
KNeighborsClassifier(metric='euclidean', n_neighbors=1)
```

Penjelasan: `grid.best_params_` adalah output dari `GridSearchCV` yang berisi kombinasi parameter paling optimal berdasarkan nilai **accuracy tertinggi** pada cross-validation.

9. Evaluasi Model KNN

```
print("\nConfusion Matrix:")  
print(confusion_matrix(y_test, y_pred))  
  
print("\nClassification Report:")  
print(classification_report(y_test, y_pred))  
  
print("Accuracy:", accuracy_score(y_test, y_pred))  
print("Precision:", precision_score(y_test, y_pred))  
print("Recall:", recall_score(y_test, y_pred))
```

Penjelasan: Pada tahap ini dilakukan **evaluasi kinerja model K-Nearest Neighbors (KNN)** setelah model dilatih dan menghasilkan prediksi. Evaluasi dilakukan menggunakan beberapa metrik penting, yaitu **Confusion Matrix, Classification Report, Accuracy, Precision, dan Recall**.

Kesimpulan

Berdasarkan seluruh rangkaian proses yang dilakukan mulai dari preprocessing data, pembersihan kolom, encoding variabel kategorikal, penyeimbangan data menggunakan SMOTE, hingga pemilihan hyperparameter terbaik dengan GridSearchCV, dapat disimpulkan bahwa algoritma **K-Nearest Neighbors (KNN)** mampu melakukan klasifikasi status kelulusan mahasiswa dengan sangat baik.

Model KNN yang digunakan memiliki konfigurasi terbaik yaitu:

- **n_neighbors = 1**
- **metric = euclidean**
- **weights = uniform**

Hasil evaluasi menunjukkan bahwa model mampu menghasilkan nilai **accuracy, precision, dan recall** yang tinggi, membuktikan bahwa model bekerja secara optimal dalam membedakan mahasiswa yang lulus tepat waktu dan tidak tepat waktu. Confusion matrix dan classification report juga menunjukkan bahwa model memiliki tingkat kesalahan prediksi yang sangat rendah.

Secara keseluruhan, implementasi KNN dalam penelitian ini berhasil memberikan performa yang sangat baik dan dapat digunakan sebagai pendekatan yang efektif untuk memprediksi status kelulusan mahasiswa berdasarkan data yang tersedia.